SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science & Engineering

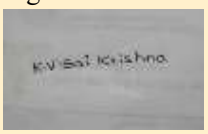
ASSESSMENT TOOL 1 – APPLICATION BASED QUESTIONS

Course Code	**CSA4315**	**Course Title**	**INTERNET PROGRAMMING**
CO Assessed	**CO1**	**Assessment**	**Assessment Tool 1: Application Based Questions**
Weightage	**40% of CIA**	**Total Marks**	**100**
CO1 Statement	*Explain the fundamentals of web architecture, client-server communication, Internet protocols, HTTP request and response messages, and configure LAMP/WAMP environments for web application development and deployment.* **(BL3)**		
Student Name	**KOPPARTHI VENKATA SAI KRISHNA**	**Reg. No.**	**192325037**
Section/Batch	**Slot D / AI&ML**	**Date**	**22-10-2026**

Code of Conduct:

I , KOPPARTHI VENKATA SAI KRISHNA, certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



16. An HTTP request contains the method POST, headers, and a request body. Explain the purpose of each component and compare the request with a GET request.

The HTTP POST Request: Method, Headers and Body

1. Introduction

Every time a browser talks to a web server it sends an HTTP request, and the way that request is written decides what the server does with it. A request that only wants to read a page is built differently from a request that submits a form. This answer looks at the second kind. An HTTP POST request is made of three parts that the question names: the method, the headers and the request body. Each part has its own job, and the request cannot work properly if any of them is wrong or missing.

The answer is organised around one running example, a student submitting a login form on **portal.example.edu** (the server is assumed to run Apache, PHP and MySQL, the LAMP setup used in this course). It first explains the purpose of the method, the headers and the body, then describes the structure of a GET request, and finally compares the two methods point by point.

2. Client-Server Communication and HTTP

The web follows the **client-server model**. The client, here the student's browser, asks for something, and the server, here Apache with PHP, holds the website and answers. They talk using **HTTP (HyperText Transfer Protocol)**, an application layer protocol that runs on top of TCP. When the address begins with https, the same messages are carried inside an encrypted TLS connection.

HTTP works on a **request and response** pattern: the client sends one request and the server returns one response. HTTP is also **stateless**, which means the server does not remember earlier requests by itself. Because of this, every request must carry everything the server needs to understand it: what is wanted (method and path), information about the message (headers), and any data being submitted (body).

Every HTTP request has the same four-part layout: a request line, header lines, an empty line, and an optional body. The rest of this answer looks at how the method, headers and body fit into that layout.

3. The POST Request and Its Components

The login form has its action set to /login.php, uses the POST method and has an email field, a password field and a hidden security token. When the student presses Login, the browser builds the message shown in Figure 1.

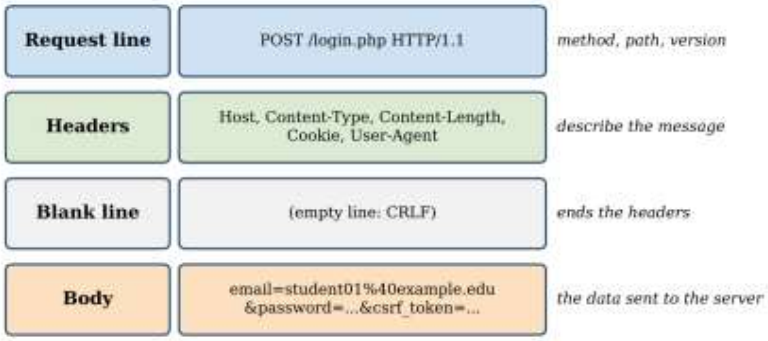


Figure 1: Structure of the HTTP POST request sent by the browser

A complete raw version of this request looks like this:

```
POST /login.php HTTP/1.1
Host: portal.example.edu
Content-Type: application/x-www-form-urlencoded
Content-Length: 78
Cookie: PHPSESSID=8f2a1c9d
User-Agent: Mozilla/5.0

email=student01%40example.edu&password=Pa55w0rd&csrf_token=a91f3c
```

3.1 The method (POST)

The **method** is the first word of the request line. It tells the server what kind of action the client wants performed on the resource named in the path. GET means "give me this resource", while **POST** means "take the data I am sending and process it". In the login example, POST tells the server that it is receiving credentials to be checked, not just asking for a page.

POST is meant for requests that change something or carry data the server must act on, such as logging in, registering, uploading a file, posting a comment or placing an order. Because the data sits in the body and the request is not meant to be repeated freely, browsers warn the user before re-sending a POST when a page is refreshed, and caches and search engines do not store POST responses.

3.2 The headers

The **headers** are name and value lines placed after the request line. They do not carry the user's data. They carry information **about** so that the server knows how to read it and how to reply. The main headers in a POST request are:

- **Host** names the website being requested. One server can host many sites, so this header lets Apache choose the correct virtual host. It is the only header HTTP/1.1 requires.
- **Content-Type** describes the format of the body, for example application/x-www-form-urlencoded for a normal form, multipart/form-data for file uploads or application/json for AJAX calls. PHP uses this value to decide how to fill $_POST.
- **Content-Length** gives the size of the body in bytes, so the server knows exactly where the message ends and how much to read.

- **Cookie** sends back cookies stored earlier, such as the session id, so that the server can recognise the same user in a stateless protocol.
- **User-Agent, Accept and Accept-Language** describe the browser and the kind of response it can handle.
- **Origin and Referer** show which page the request came from, which helps the server reject forms posted from other websites (CSRF protection).

3.3 The request body

The **body** is the part after the empty line, and it holds the actual data being sent to the server. For a standard form the browser writes each named field as a key and value pair, joins the pairs with the & symbol, and percent-encodes special characters so they cannot be mistaken for separators (the @ sign becomes %40 and a space becomes %20). In our example the body holds three pairs: the email, the password and the hidden CSRF token.

The body can be much larger than a URL can safely be, and it can hold any type of data: form fields, JSON, XML or the binary content of an uploaded file. It is not shown in the address bar, not saved in browser history and not stored in bookmarks. Over HTTPS it is also encrypted in transit, which is why a password must always be sent in a body and never in a URL.

4. The GET Request

A GET request is used to **retrieve** a resource such as a web page, an image or a search result. It has the same overall layout (request line, headers, blank line), but it has **no body**. Any extra data is attached to the URL as a **query string**: a ? after the path, followed by name=value pairs joined with &. Figure 2 shows a search request.

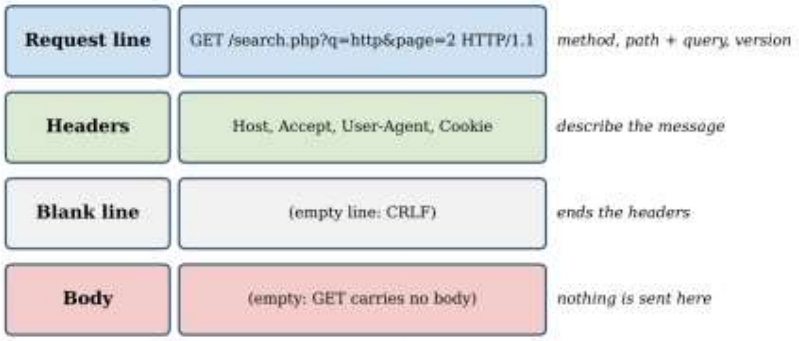


Figure 2: Structure of an HTTP GET request

Because the data is part of the URL, a GET request can be bookmarked, shared as a link, stored in the browser history and cached. It is safe and idempotent, which means that sending it many times does not change anything on the server, so the browser can repeat it freely. For the same reason it must never be used to send passwords or to change data.

5. Comparison of POST and GET

Figure 3 shows where the data travels in each method, and the table below compares them on the points that matter in practice.

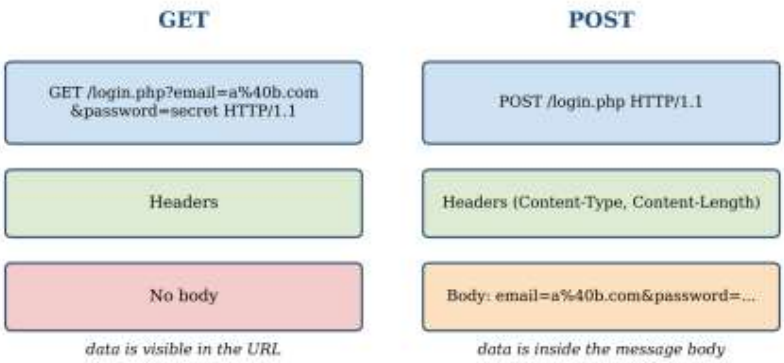


Figure 3: Position of the submitted data in a GET and a POST request

Feature	GET	POST
Purpose	Retrieve a resource (read)	Send data for processing (create or change)
Where data is placed	In the URL as a query string	In the message body
Request body	None	Present (form data, JSON, files, etc.)
Visibility	Shown in address bar, history, bookmarks and server logs	Not shown in the URL; encrypted in transit when HTTPS is used
Data size	Limited by the URL length (about 2,000 characters in practice)	No practical limit set by the protocol; the server may set one
Data types	Text only (URL-encoded)	Text, JSON, XML and binary data (file upload)
Content-Type / Content-Length headers	Not needed	Required to describe the body
Security for sensitive data	Unsuitable: leaks through history, logs and Referer header	Preferred, together with HTTPS and CSRF tokens
Caching and bookmarking	Can be cached and bookmarked	Normally not cached or bookmarked
Idempotent and safe	Yes, repeating it has no side effects	No, repeating it may create duplicate records
Browser refresh / back	Re-sent silently	Browser asks for confirmation before re-sending
Typical use	Opening a page, search, filters, links	Login, registration, uploads, checkout, comments

Table 1: Comparison of GET and POST requests

6. How the Components Work Together

Figure 4 follows the complete exchange for the login example. The POST request carries the credentials in its body and describes them in its headers. After the server checks them, it replies with a 302 redirect and a Set-Cookie header, and the browser then follows up with a GET request for the dashboard. This is the Post/Redirect/Get pattern: POST is used to submit the

data, and GET is used to display the result, so refreshing the dashboard never submits the form again.

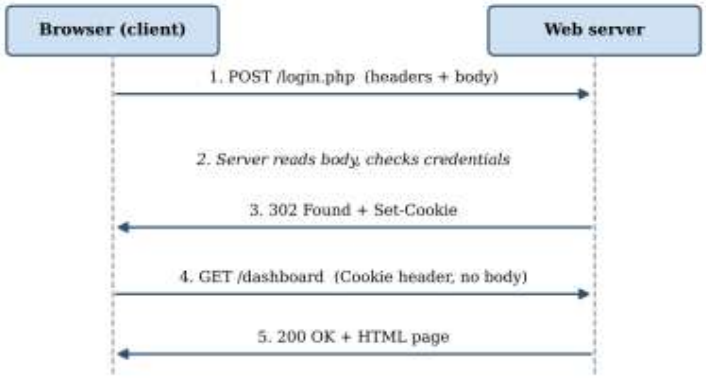


Figure 4: POST followed by a redirect and a GET request

7. Choosing Between GET and POST, and Security Points

- **Use GET** when the request only reads data, when the result should be bookmarkable or shareable, and when the parameters are not sensitive (search terms, page numbers, filters).
- **Use POST** when the request changes data on the server, carries sensitive values such as passwords, sends large data or uploads files.
- **Do not treat POST as encryption.** A POST body is hidden from the URL but is still readable on the network unless HTTPS is used, so HTTPS is needed for every login form.
- **Validate again on the server.** Client-side checks can be bypassed, so the server must validate the body, use prepared statements against SQL injection, and check a CSRF token because a POST changes state.
- **Keep secrets out of URLs.** Anything in a GET query string can end up in browser history, server logs and the Referer header sent to other sites.

8. Conclusion

An HTTP POST request is built from three cooperating parts. The method states the intention, which is to submit data for processing. The headers describe the message, including the host, the format and size of the body and the session cookie. The body carries the data itself, hidden from the URL and free from the length limits of a query string. A GET request has the same request line and headers but no body, so its data must travel in the URL, which makes it ideal for reading and sharing but unsuitable for sensitive or state-changing actions. Understanding this difference explains why login forms, uploads and checkouts use POST over HTTPS, while searches and page links use GET.

Evaluation Rubric

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
HTTP basics and message format	15	Clear, correct account of client-server model, statelessness and message structure	Mostly correct, minor gaps in HTTP basics	Partly correct, some points missing	Mostly incorrect or very brief
POST components explained	30	Method, headers and body each explained with purpose and example	Main components covered, few details missing	Only some components, limited explanation	Components unclear
GET and POST comparison	30	Detailed, accurate comparison across data placement, size, security, caching and idempotency	Main differences covered with some reasons	Few differences, little explanation	Comparison barely given
Security and usage	10	Reasons for choosing GET or POST explained with HTTPS, CSRF and logging	Key points listed with some reasons	Mentioned without explanation	Little or no discussion
Presentation and diagrams	15	Well organised, clear language, relevant figures with captions	Organised, few figures or small errors	Average structure, unclear in places	Poorly organised